FaceMango

How FaceMango works

The architecture, tools and workflows behind the most personal social network ever built. In plain English, with pictures.

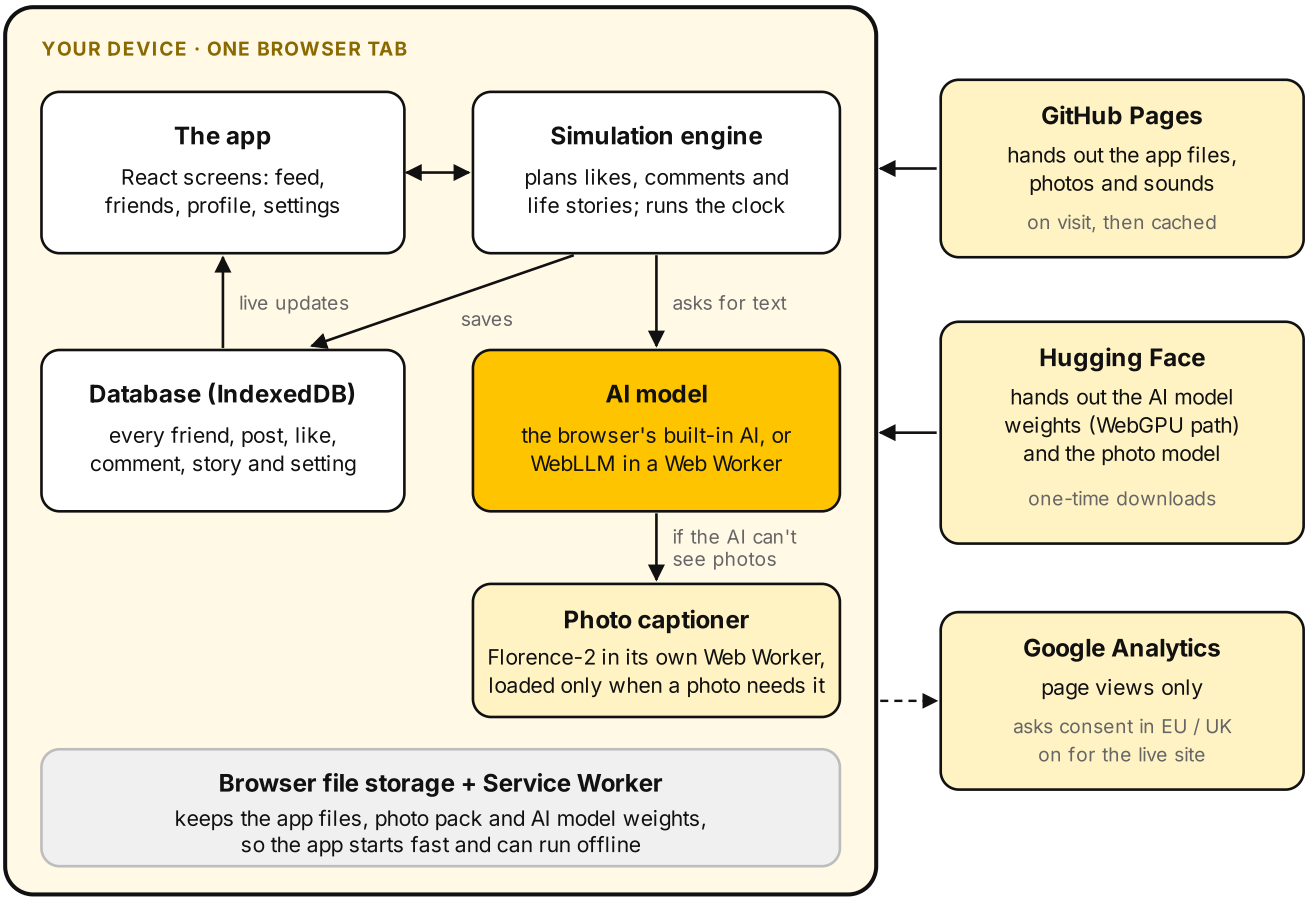
> **In one sentence:** FaceMango is a website that runs entirely inside your browser. A small AI model on your own device plays your friends, a database on your own device remembers everything, and no server ever sees what you post.

What is inside

1	**The big picture**	What lives on your device and what comes from outside
2	**How the code is organised**	Four layers and who talks to whom
3	**Frameworks and libraries**	The toolbox, and what each tool does for the app
4–6	**The AI layer**	Three tiers, a waiting line, checking answers, and seeing photos
7–8	**The simulation engine**	Planning likes and comments, and friends' life stories
9–10	**Workflows**	First run, posting, and coming back after a break
11	**Your data**	The database and the friends reset
12	**Screens, offline use and privacy**	Routes, the app shell, PWA and the network rules
13	**Building, testing and shipping**	From a code change to face.mangoidiots.com
14	**Where to find things**	Code map, glossary and current status

1 The big picture

FaceMango has no back end. Almost everything happens inside one browser tab on your own phone or computer. A few outside services only hand over files, once.



✗ Your posts, photos, friends and wellbeing data never leave the device.

Figure 1 · Everything inside the big box runs locally. The three boxes outside only send things *in*.

No back end

There is no FaceMango server, account or cloud storage. GitHub Pages is just a file shelf: it serves the same static files to everyone.

The friends are AI

A language model running on your device writes every comment, friend post and life story. The like counts come from simple rules, not from the model.

Your data stays in the browser

IndexedDB holds it all. You can export a zip (or have one saved weekly to a folder) and restore it on another device, but nothing syncs by itself.

It installs like an app

A Progressive Web App (PWA): add it to your home screen, and after the first visit it can start offline.

2 How the code is organised

The code is written in TypeScript and sorted into four layers. Each layer only talks to the one below it.

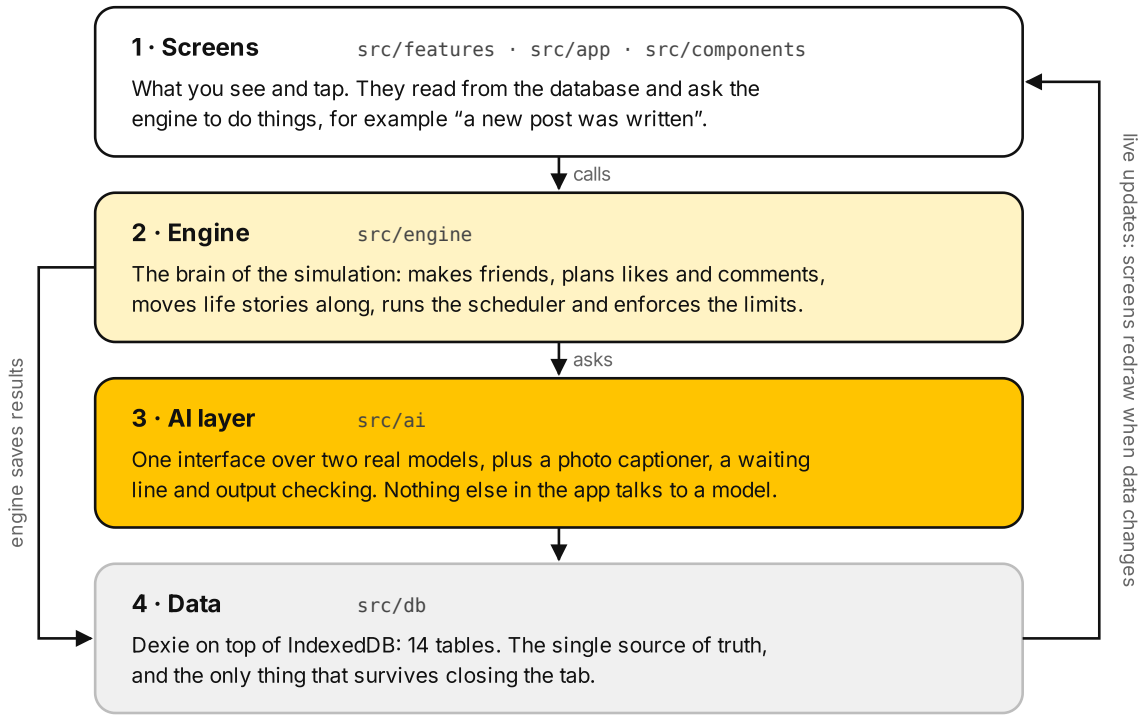


Figure 2 · Calls go down; results come back up through the database, not through function return values.

Three design choices worth knowing

- **Screens never wait for the AI.** The engine writes a comment into the database and the screens, which are watching the database with Dexie *live queries* (used across dozens of screens), redraw by themselves. A slow model never freezes the UI.
- **One door to the model.** Everything goes through `src/ai`, so the rest of the app does not know or care whether Chrome's built-in AI, WebLLM or the photo captioner is answering. Swapping or adding a model touches one folder.
- **Rules live in one place.** Hard limits (at most 1,000,000 likes and 100 comments per post) sit in `src/engine/limits.ts` and are unit-tested, so no setting or bug can break them.

> **Threads, in short.** The screens and the engine's scheduler loop run on the browser's main thread. The WebLLM model and the photo captioner each run in their own **Web Worker**, so heavy GPU work never blocks scrolling.

3 Frameworks and libraries

All open source, all compatible with the app's AGPL-3.0 licence. Here is the toolbox and what each tool does for FaceMango.

TOOL	WHAT IT DOES HERE
Language, build and screens	
TypeScript 6	JavaScript with types, in strict mode, so many mistakes are caught early.
Vite 8	Dev server with instant reload, and the bundler that makes the production files.
React 19	Builds every screen from small reusable components.
React Router 8	Page addresses (`/friends` , `/post/:id`) and lazy loading of screens.
Look and feel	
MUI 9 + Emotion	Material UI: ready-made buttons, dialogs, navigation and the FaceMango theme (white, mango yellow `#FFC400`).
Motion	Smooth animations: page transitions, the like button, lists appearing.
react-virtuoso	Draws only the posts on screen, so a very long feed stays fast.
dotLottie, confetti, Howler	Animated reaction emoji, celebration confetti and short UI sounds.
MUI X Charts, @use-gesture	Wellbeing charts; swipe, long-press and pinch. Fonts are served by the app itself.
Data and state	
Dexie 4 + dexie-react-hooks	Friendly layer over IndexedDB; live queries redraw screens when data changes.
Zustand 5	Small in-memory stores: AI status, setup progress, “Priya is typing…”, UI state.
Zod 4	Describes the shape of data, and checks the model's JSON answers against it.
React Hook Form, date-fns	The signup form; “5m”, “Yesterday at 4:12 PM” and the dates behind Memories.
AI	
Prompt API	The browser's own model (Chrome: Gemini Nano, Edge: Phi-4-mini). No library.
@mlc-ai/web-llm	Runs open models (Qwen3, Gemma 3) on the GPU via WebGPU, in a Web Worker.
Transformers.js + ONNX Runtime	Runs the small Florence-2 photo model in its own Web Worker, so friends can comment on what is in a photo even when the main AI can't see images.
Media and offline	
DiceBear Personas	Draws each person's illustrated avatar from traits the AI picks.
browser-image-compression	Shrinks your photo to WebP on the device before saving it.
vite-plugin-pwa + Workbox	Installable site, offline caching, and the “update available” prompt.
Quality and delivery	
Vitest, Testing Library	Fast unit and component tests (`fake-indexeddb` stands in for the database).
Playwright	Drives real browsers end to end: desktop Chrome, a Pixel 7 and an iPhone 15 (WebKit). It also prints this guide. ESLint and Prettier keep the code tidy.
GitHub Actions + Pages	Test, build and publish automatically; Cloudflare provides the DNS for `face.mangoidiots.com` .

4 The AI layer: three tiers

Devices differ a lot, so FaceMango looks for the best AI available and falls back step by step. There is deliberately no “fake AI” mode: with no model, the app explains why and stops.

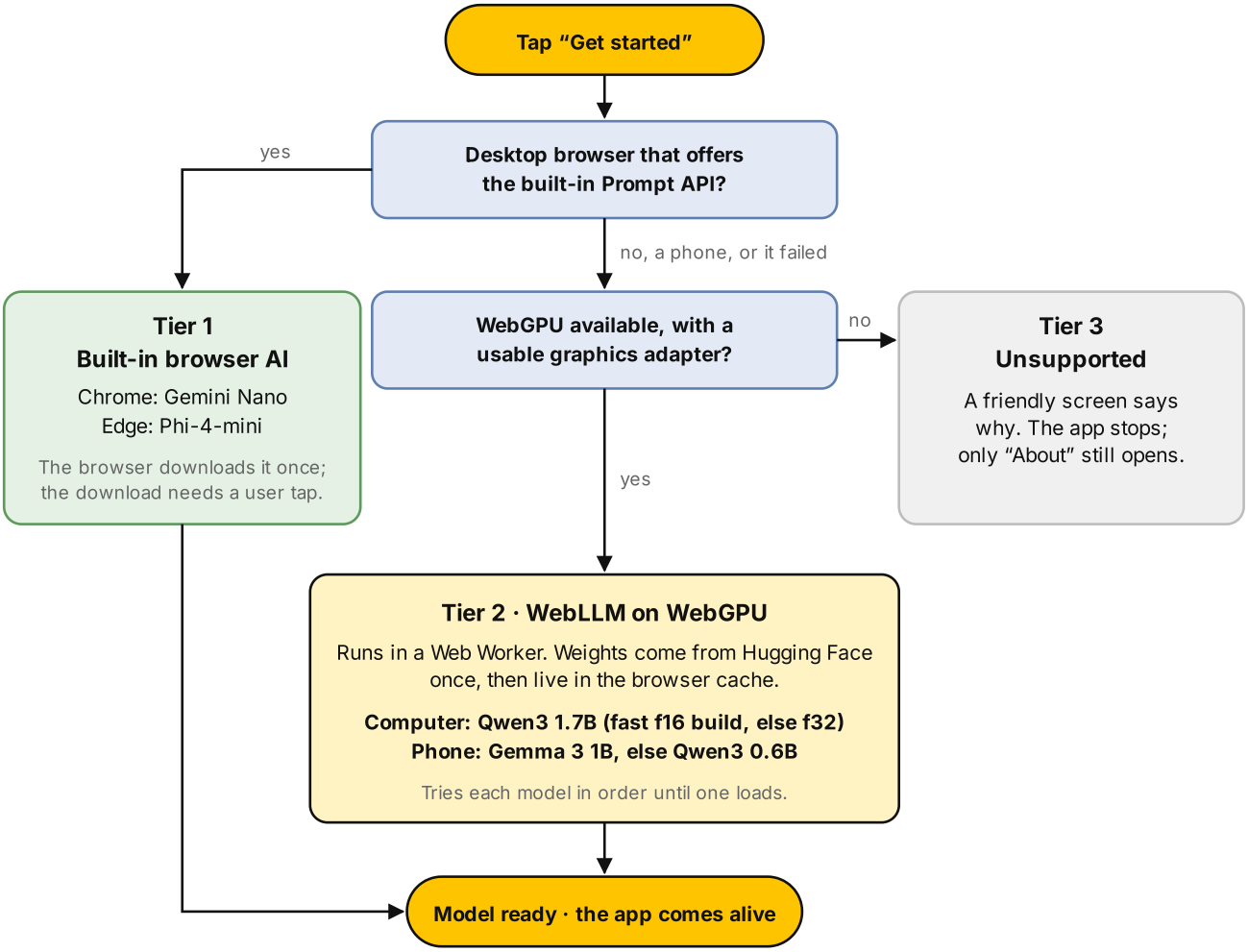


Figure 3 · Choosing the AI tier (`src/ai/detect.ts`, `src/ai/service.ts`)

Why start on a tap?

Browsers only allow the big model download after a user gesture, so “Get started” kicks it off. Later visits start silently.

Phones skip Tier 1

Mobile browsers go straight to WebGPU. If Tier 1 fails on a computer, it falls through to WebGPU too.

One shared interface

Both tiers implement `AIProvider`: `generate`, `generateJSON`, `stream` and optionally `describeImage`. Without it, the photo captioner steps in (section 6).

5 Keeping the model polite

A small on-device model can do one job at a time, and small models sometimes get the format wrong. Two safeguards handle that.

A waiting line with a fast lane

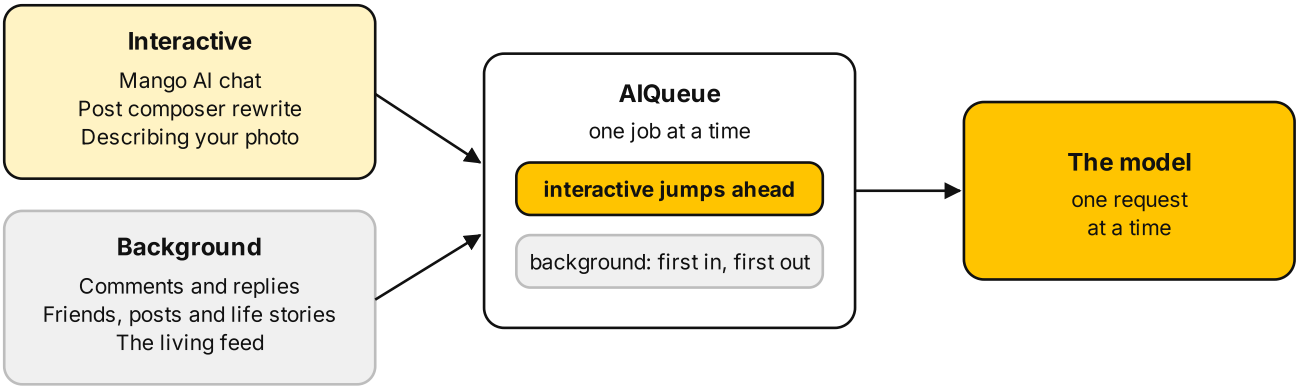


Figure 4 · If you ask Mango AI something while comments are being written, your request goes next.

Checking the model's homework

For friends, posts, stories and comments the app needs structured answers (JSON), not free text. Every such call is wrapped in the same loop:

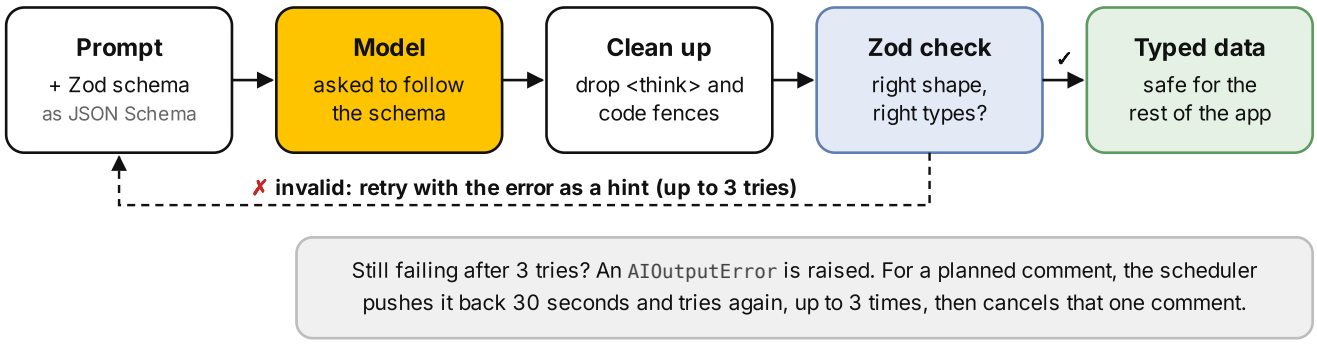


Figure 5 · `generateValidated` in `src/ai/json.ts`. No unchecked model output ever reaches the database.

> **Streaming with a safety net.** Mango AI chat shows words as they arrive. If streaming fails or returns nothing, the app asks once for the whole reply and shows the error in Settings → On-device AI, instead of leaving a blank bubble.

6 Seeing your photos

Friends comment on what is *in* your photo, not just on your words. Many text models can't see pictures, so when needed a second, much smaller model first describes the photo in one sentence.

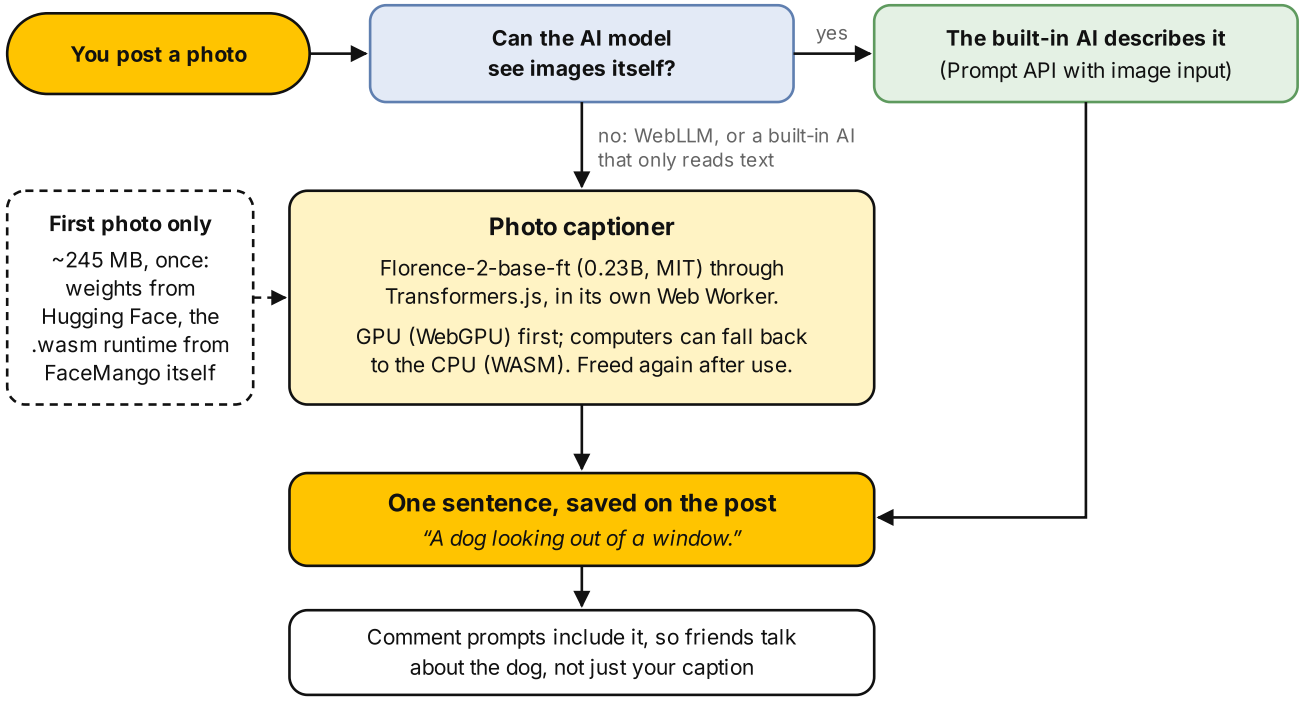


Figure 6 · Describing a photo (`src/engine/describe.ts`, `src/ai/captioner.ts`). The photo never leaves the device; only model files come in.

Only when needed

Nothing downloads until your first photo post, and only where the AI can't see images. Each photo is described once, even if you post it again.

Polite with the GPU

The caption waits its turn in the same AI queue (fast lane), so it never runs at the same time as the text model. Then the worker shuts down to free memory.

Careful on phones

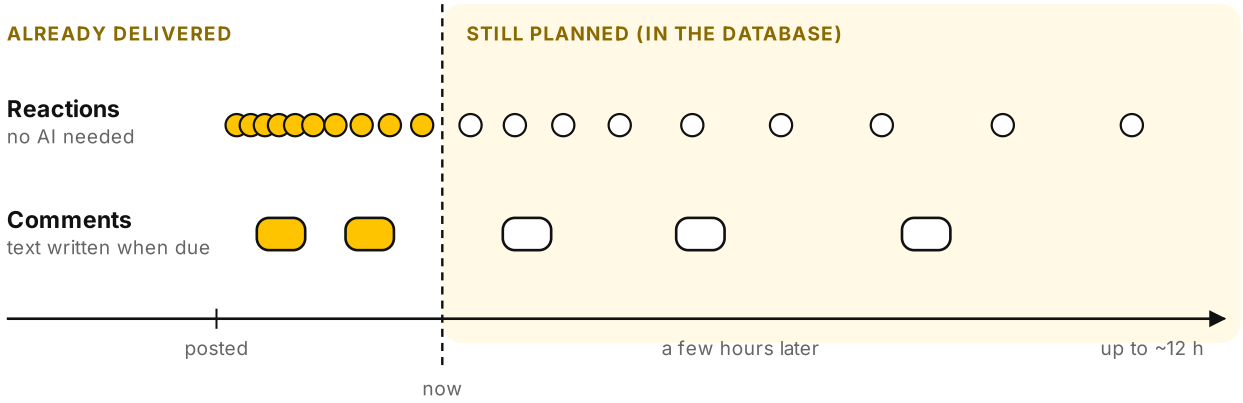
Phones that report under 4 GB of memory skip it, and so does a phone where it was cut off twice. Comments then use your words only.

Your switch, and the outside world

- **Settings → On-device AI:** "Describe my photos so friends can comment on them" is on by default. The same panel shows progress and has a **Delete photo model** button.
- **No new third party.** Transformers.js would normally fetch ONNX Runtime from a CDN. FaceMango ships its own copy as a hashed `.wasm` file, so the only outside servers stay Hugging Face and Google Analytics.
- **Licences:** Transformers.js (Apache-2.0), ONNX Runtime Web (MIT) and Florence-2 (MIT), all compatible with AGPL-3.0.

7 The simulation engine

The core trick: **plan first, write later.** The moment you post, the engine decides the whole story of that post. Only the words are produced when each moment arrives.



Reactions bunch up early with random bursts and lulls, like real attention.
Close the tab and the plan keeps its place; see “Coming back after a break”.

Figure 7 · One post's engagement plan. Each dot or bubble is a row in the `events` table.

How a plan is made (the defaults)

- **Comments:** 0 to 9, skewed towards the low end. Never above 100.
- **Likes:** comments × 3 to 5, at least 3. Never above 1,000,000.
- **Who:** friends are picked by closeness and activity; public profiles fill in the rest, shown as “and 1,234 others”.
- **Mood:** 60% good, 25% appreciative, 10% nonsense, 5% critical, at most 2 critical per post. Fans, neutrals and critics tilt the mix.
- **@mentioned** friends always respond.
- **Your first post** gets a warm early burst.
- **Friends' own posts** get lighter engagement (about 45%).
- A **speed** setting squeezes the timeline (0.25× to 60×).

The scheduler: a clock that never stops

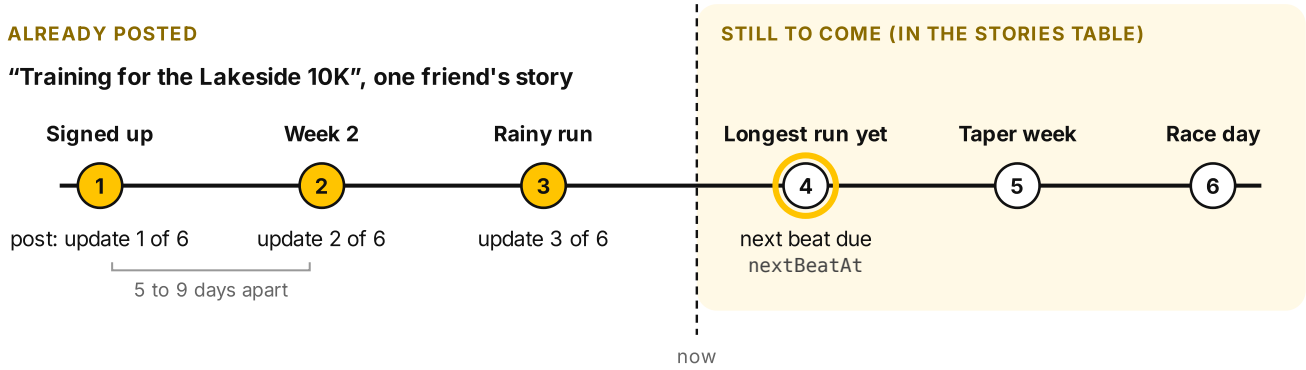
Starts with the app shell; wakes every 2.5 s (0.2 s while busy).

1. **Apply due reactions.** Plain database updates, no AI.
2. **Show “is writing a comment…”** for comments due in the next 20 s.
3. **Write due comments** in batches of 4 for the same post, your own posts first.
4. **Friends post** on their own every 40 to 150 minutes; about 3 in 10 of these posts move a **life story** along (next page).
5. **Friend requests** arrive from public profiles now and then.

Event life cycle. Every planned item is `planned` → `applied` when delivered, or `cancelled` if the post is gone, the person became a former friend, or the AI failed three times.

8 Friends' life stories

Friends don't only post one-offs. Some live through small storylines that unfold over days and weeks (a new job, race training, a new pet) told one post at a time.



Each beat is one friend post. The earlier beats go into the prompt, so posts refer back (“Week 3 of training…”).

Figure 8 · A story arc. The plan is data, so even a small model stays consistent from one post to the next.

How a story post happens

1. A friend post is due, and about 3 in 10 are story posts.
2. If a story's next beat is due, the **most overdue beat** is posted by its owner.
3. Otherwise a **new story starts**, if there is room: the AI fills the arc's blanks as Zod-checked JSON, and beat 1 is posted.
4. No room? It is an ordinary one-off post.

The house rules

- At most one active story per friend, and a third of friends at once.
- New stories start at least 16 hours apart.
- A friend rests 7 days after a story ends.
- The speed setting squeezes the gaps, like everything else.
- A few posts in the first-run feed already begin stories.

Curated in code

12 arcs in `storyTemplates.ts`, each with 4 to 6 beats in a fixed order. Family-friendly: no tragedy, illness or breakups; mild setbacks only.

Filled in by the AI

Only the specifics: a race, a pet's name, a dish, a city. Then the words of each post, written when its beat comes up.

Kept as data

The `stories` table: title, beats, current beat, next beat time, status (active, completed, abandoned) and links to the posts.

Where you see them. Post cards say “Training for the Lakeside 10K · update 3 of 6”, commenters may ask how it is going, and profiles list ongoing, completed and ended stories. A friends reset ends the old friends' unfinished ones.

9 Workflow: first run

The first visit is the heaviest moment: a model has to arrive, and a whole social circle has to be invented. The app overlaps the two, and tells you plainly what is happening.

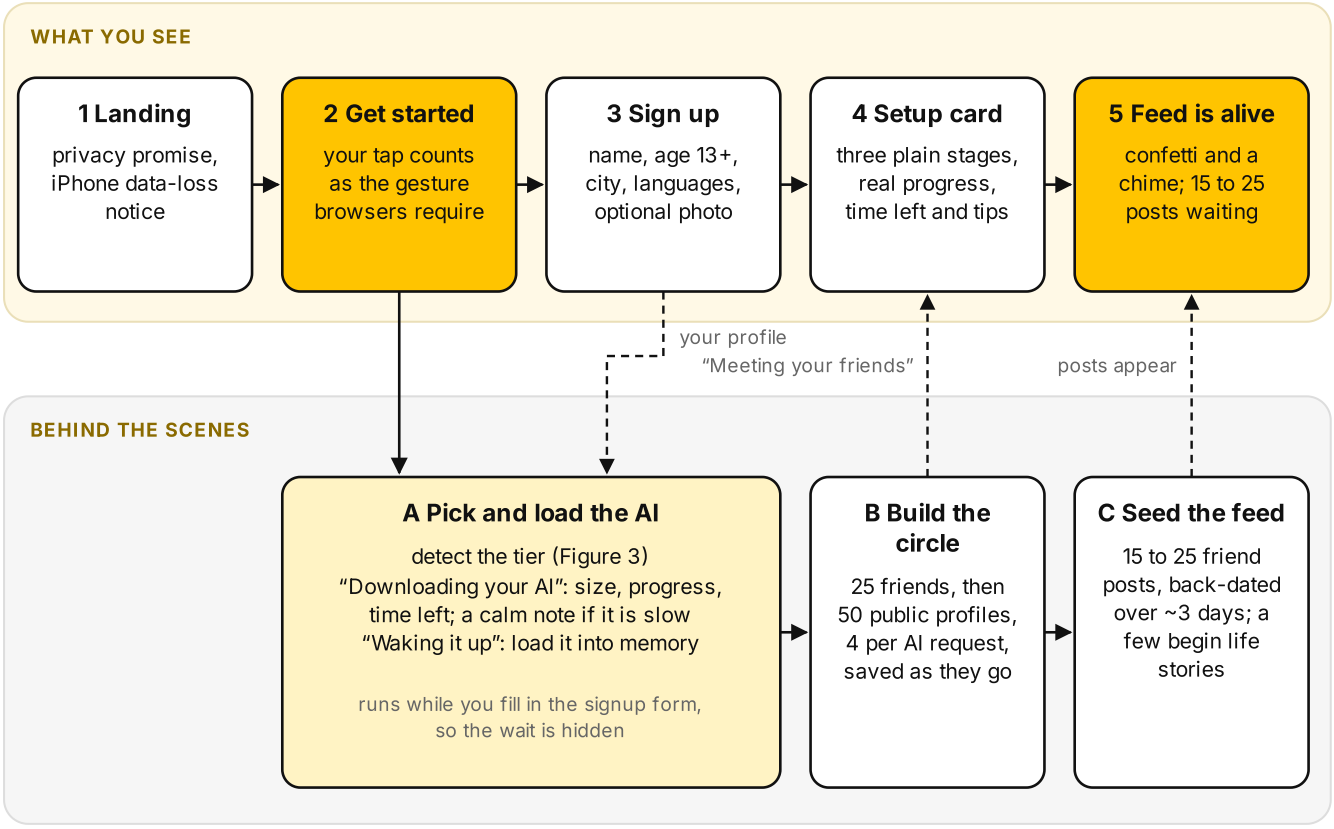


Figure 9 · Solid arrows: order of work. Dashed arrows: progress shown on screen, or data feeding the screen.

Resumable by design

Each stage tops up what is stored. Close the tab at friend 14 and it carries on from friend 14. `setupComplete` is set only at the end.

Post while you wait

A post written before the friends arrive is kept in a waiting list and gets its likes and comments as soon as setup finishes.

Where it lives in code

`features/onboarding`, the setup card in `features/ai`, progress in `ai/progress.ts`, world-building in `engine/onboarding.ts`.

When something goes wrong

No usable AI

Neither the built-in AI nor WebGPU works: the Unsupported screen explains why, and the app stops there.

A model will not load

The next model in the list is tried. If all fail, the setup card shows the reason and a clear Retry button.

The AI errs mid-setup

Setup pauses with a short message and resumes the next time you open FaceMango, from where it stopped.

10 Workflow: you post something

From the tap on “Post” to the first heart arriving, here is who does what.

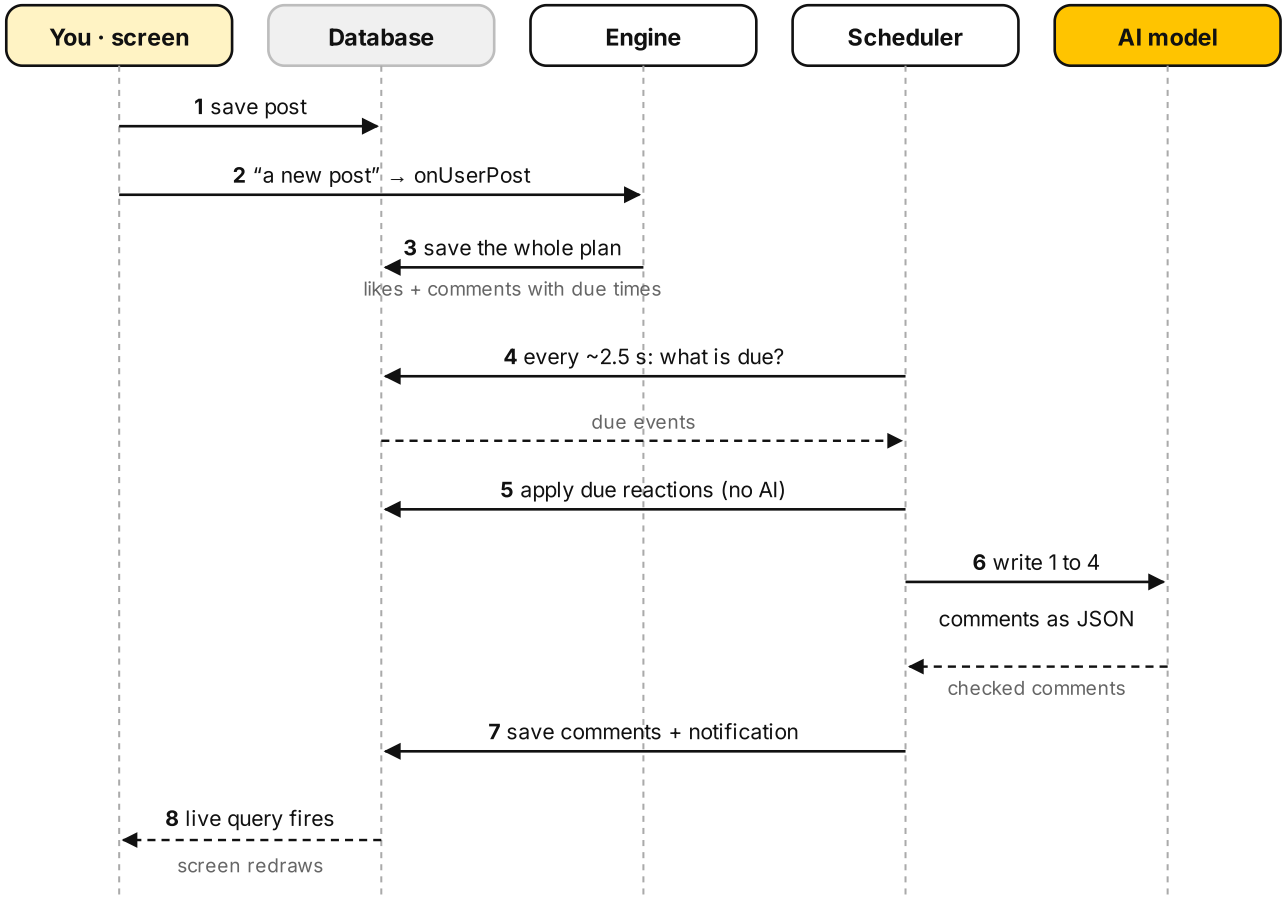


Figure 10 · Dashed arrows are replies. Steps 4 to 7 repeat for as long as events are due.

While comments are being written

Because comments on your own posts are planned ahead, the app knows who is about to comment and shows “Priya is writing a comment…” for events due in the next 20 seconds.

With a photo

Alongside step 2, the photo is described once (section 6) and the sentence is saved on the post, so the comments in step 6 can mention what is in it.

When you comment or reply, the engine plans one friend reply: about 65% likely to answer a reply to them, 75% likely to answer you on their own post, usually within 1 to 8 minutes.

Coming back after a break

There is no server to keep the story going while the tab is closed, so the app catches up instead:

1. Planned items whose time passed stay **planned** in the database.
2. When you return, the scheduler finds them all due. Reactions are applied with their **original timestamps**, so the feed looks as if it had been live all along.
3. If a lot happened, a “While you were away” card sums it up: for example *14 reactions, 3 comments*.

11 Your data

Everything is stored in a single IndexedDB database named `facemango`, managed with Dexie. These are its 14 tables, in plain terms.

People

profile you

personas every friend, public profile and former friend: personality, interests, avatar traits

Content

posts yours and your friends'

comments **reactions**

media photos and avatars, stored as blobs

stories friends' life stories and their beats

The clock

events the planned future: reactions, comments, replies, friend requests

notifications the bell and Activity

You, privately

usageSessions **moods**

Wellbeing data, never sent anywhere

chat Mango AI history

Settings

settings your choices

meta internal flags: setup done, next friend post

Rules for changing it

Never edit a shipped schema version. Add a new Dexie version (now at v3, which added `stories`). That keeps every existing user's data safe.

Resetting your friends



12 Screens, offline use and privacy

How the app is wired

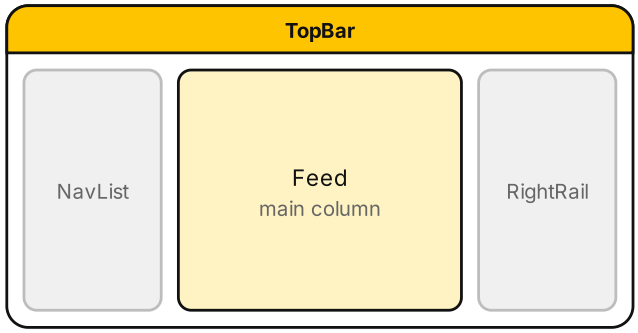
`main.tsx` mounts `App`, which sets up the theme, animations and the router. `RootLayout` wraps every page: it blocks the app with the Unsupported screen if no AI exists, sends analytics page views, and hosts toasts and the update prompt.

```
Open to visitors
  /welcome          landing page
  /signup           signup form
  /about /credits   info pages

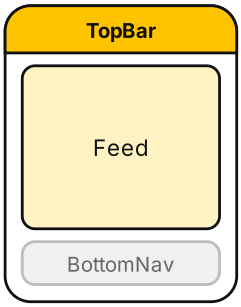
Needs a profile (inside AppShell)
  /                 home feed
  /friends          friends + requests
  /profile/:id      profile, life stories
  /post/:id         one post
  /assistant        Mango AI
  /memories         on this day
  /notifications    Activity
  /photos           photo grid
  /wellbeing        usage dashboard
  /settings         + /settings/advanced
  /settings/developer  real-model test
  /menu             mobile menu
```

Most screens are loaded on demand (`lazy`), so the first page opens quickly. Guards send visitors without a profile to `/welcome`, and returning users straight to the feed.

One app, two layouts



Computer: three columns



Phone: one column and a bottom bar (`AppShell`)

Installable and offline

- **Service worker** (Workbox, via `vite-plugin-pwa`) caches the app files and the photos you view. WebLLM and the photo model's runtime are cached the first time they are used.
- **"Update available"** prompt: a new version waits until you accept it, so a refresh never interrupts you.
- **Deep links** such as `/about` work through a `404.html` copy of the app that GitHub Pages serves.
- **iPhone note:** Safari can erase site data after 7 idle days unless the app is on the Home Screen. FaceMango warns about this and nudges you to add it to the Home Screen.

Privacy, in layers

- **By design:** no server and no account; posts, photos and wellbeing data have no code path to the network.
- **By the browser:** a strict *Content-Security-Policy* only allows connections to the app itself, Hugging Face and Google Analytics.
- **By self-hosting:** fonts, emoji, sounds, model libraries and the photo model's runtime are copied into the app (`npm run models`, the build) instead of loaded from a CDN.
- **Analytics:** on for the live site (the `GA_ID` build variable). It sends page views with route patterns like `/post/:id`, never ids or text, and waits for consent in the EU and UK.

13 Building, testing and shipping

Every push to `main` is checked by robots and, if everything passes, published to the live site.

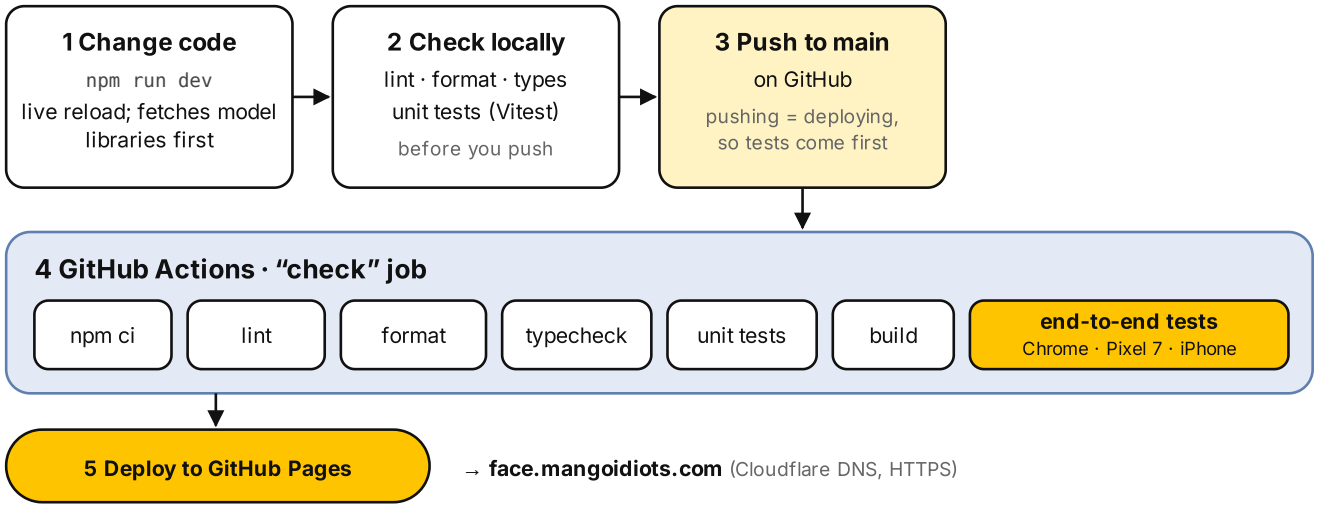


Figure 11 · `.github/workflows/deploy.yml`. On pull requests the same checks run, but nothing is deployed.

Three kinds of tests

- **Unit tests** (`npm test`): Vitest in Node with a fake database. They cover the planner, limits, queue, JSON checks, life stories and the photo captioner. The database is cleared before each test.
- **End-to-end tests** (`npm run test:e2e`): Playwright drives the real built app on desktop and phone shapes, using a **mock model** that only exists in the test build (`VITE_MOCK_AI=1`).
- **Real-model checks:** a mock cannot judge writing quality. The **Real-model test** page (Advanced settings → Developer) runs the real prompts through the loaded model in Chrome or Edge and reports validity, retries, speed and tone mix, storing nothing.

House rules (in `CLAUDE.md`)

- Hard caps: 1,000,000 likes and 100 comments per post.
- User content and wellbeing data never go over the network.
- All AI calls go through `src/ai`, with Zod-checked JSON.
- Never edit a shipped database version; add a new one.
- No non-AI mode; the mock model is test-only.
- Lint enforces React Compiler rules: no `Date.now()` during render, use `useNow()`.

Everyday commands

`npm run dev`	Start the dev server at `localhost:5173`. It first fetches the WebLLM model libraries into `public/models/` (git-ignored), so the app serves them itself.
`npm run build`	Type-check and make the production files in `dist/`, plus the `404.html` fallback and the PWA service worker.
`npm test`	Run all unit tests once (`test:watch` keeps them running).
`npm run test:e2e`	Build the mock-model version into `dist-e2e/` and run the Playwright tests against it.
`npm run lint · format`	Find problems with ESLint; tidy the code with Prettier.
`npm run docs:pdf`	Print this guide from its source, `docs/how-it-works/how-it-works.html`, into `public/`.

14 Where to find things

FOLDER	WHAT IS INSIDE
`src/app/`	App root, routes, guards, theme, and the responsive shell (`shell/`).
`src/features/`	One folder per screen: feed, compose, friends, profile, assistant, memories, activity, photos, wellbeing, settings, onboarding, ai (setup card), developer.
`src/engine/`	`planner` (the plan), `scheduler` (the clock), `onboarding` , `userPosts` , `friendPosts` , `stories` + `storyTemplates` , `describe` , `personas` , `prompts` , `limits` , `config` , …
`src/ai/`	`detect` (choose a tier), `service` , `queue` , `json` (validate, retry), `progress` , `promptApi` , `webllm` + worker, `captioner` + worker, `mockProvider` .
`src/db/`	Dexie schema and versions (`db.ts`), record types (`types.ts`), data actions.
`src/lib/`	Helpers: analytics, time-ago, sounds, usage and wellbeing, image handling, platform checks.
`scripts/`	Asset scripts (model libraries, emoji, sounds, photo pack) and this guide's printer.
`e2e/`	Playwright tests: onboarding, feed, sections, developer page.
`SPEC.md, docs/`	The spec; `DEVELOPMENT.md` (rules), `STATUS.md` (open items), `how-it-works/` (this guide).

Glossary

Persona	A simulated person: name, city, interests, writing style, stance, activity, closeness.
Planned event	A scheduled future like, comment, reply or request, stored with a due time.
Life story / beat	A friend's storyline from a curated arc / one step of it, told in one post.
Prompt API	A browser feature that gives web pages access to a built-in AI model.
WebGPU / WebLLM	WebGPU lets web pages use the graphics card; WebLLM uses it to run open AI models.
Photo captioner	Florence-2, a small model that turns a photo into one sentence (via Transformers.js).
Web Worker	A background thread in the browser, so heavy work does not freeze the screen.
IndexedDB / Dexie	The browser's built-in database / the friendly library on top of it.
Live query	A database read that re-runs by itself when the data changes, so screens update.
Zod	A library that describes the shape of data and checks real data against it.
PWA / Service worker	A website that installs like an app / the background script that makes it work offline.
Content-Security-Policy	A rule set telling the browser which servers the page may contact.

Where the project stands

Built and live

M1 to M7, plus life stories, photo-aware comments on every tier, the new first-run wait, a developer test page and a 251-photo pack. Analytics on.

Open items

Re-test Mango AI with the test page; try the new features in real browsers.

Parked

M8 realistic faces (no suitably licensed small image model yet); Messenger-lite.